CONTENTS

Advanced AI evaluation Dave Ebbelaar

The Three Levels of LLM Evals: Unit Tests, Aligned Judges & A/B Testing

Build an evaluation-driven workflow — assertions on every change, human-aligned judges every week, and A/B tests on major releases.

For engineers building production LLM applications who want a systematic way to improve them instead of guessing · 27 July 2026

Watch the source video | Read the condensed version | Read the highlights

BEFORE YOU START

- Comfortable writing Python, including functions and exceptions
- You have built at least one LLM workflow with a prompt and structured output
- Familiarity with the idea of tracing or logging requests in production

Overview

Reports suggest a large share of AI projects fail to reach production — Gartner has projected that 40% of agentic AI projects will be cancelled by 2027, and an MIT figure of 95% of generative AI pilots failing has circulated widely. Whether or not those exact numbers hold, the pattern behind them is real: teams build something that works on the happy path, ship it, and then have no systematic way to find out what is going wrong or to fix it without breaking something else.

This lesson lays out the engineering answer. It starts with the three core challenges that make LLM development different from ordinary software — you do not understand the data flowing through your system, you cannot fully specify what you want in a prompt, and behaviour is inconsistent across near-identical inputs. It then presents evaluations organised into three levels of increasing cost: unit tests you run on every change, human and model evaluation you run weekly, and A/B tests you run on major releases.

The centrepiece is the alignment process: how to build an LLM judge that actually agrees with a domain expert, measured by an agreement score you drive upward through iteration. This is the step nearly everyone skips in favour of an off-the-shelf tool with a pretty dashboard, and skipping it is what produces confident metrics that mean nothing.

KEY TAKEAWAYS

- ✓ Three core challenges define LLM development: not understanding your data at scale, the gap between what you want and what you can specify, and inconsistent behaviour on similar inputs.
- ✓ An eval is a systematic measurement of quality — a single metric for a specific aspect. One system has many evals.
- ✓ Evals are operationalised three ways: background monitoring, real-time guardrails, and improvement tooling for labelling and fine-tuning data.
- ✓ The analyze → measure → improve cycle is the working loop: collect and categorise failures, turn them into metrics, then change prompts, models, or architecture.
- ✓ Level 1 is unit tests — fast, cheap assertions run on every code and prompt change.
- ✓ Level 2 is human and model evaluation — systematic review run weekly or bi-weekly. Humans must come first; the LLM judge is aligned to them, never the reverse.
- ✓ Level 3 is A/B testing with real users, reserved for major releases because of its cost.
- ✓ The agreement score between human and model critique is the metric you optimise; meta-prompting can rewrite the judge prompt to raise it.
- ✓ Reference-based metrics compare to a known correct answer; reference-free metrics score qualities like tone or safety where many answers are valid.
- ✓ The four common mistakes are tool-first thinking, generic-metrics obsession, avoiding your data, and unaligned judges.

01 The questions evaluations exist to answer

2:04

> Every engineer building on LLMs eventually hits a wall of questions that logs alone cannot answer.

Start with the questions, because they make the need concrete. What happens if I adjust the system prompt? How do I know if my RAG pipeline is working correctly? Is the classification step right? Beyond factual accuracy, is the tone of voice on point? How will this behave on real user inputs we have not seen? Are we catching unsafe or problematic outputs? How do I know when performance is degrading in production? Is the system robust enough to flag prompt injections?

If you build a workflow, put it in production, and let it run, you have no principled way to answer any of these. A tracing platform helps — you can inspect what happened after something goes wrong — but that only lets you react to issues as they surface.

The deeper problem is regression. When you change a system prompt to fix one case, how do you know you have not opened holes elsewhere, for other cases and other data flowing through the system? Reactive log-reading cannot tell you that, because you would have to notice the new failure first. Evaluations give you a fixed set of cases you re-run after every change, which is what converts 'I fixed it' into 'I fixed it and nothing else broke'.

KEY POINTS

- → Tracing platforms let you diagnose issues reactively, after they surface — necessary but not sufficient.
- → The hard question is regression: does fixing one case break others you are not watching?
- → LLMs are non-deterministic, context-sensitive, and a response can be factually correct while the tone is wrong.
- → A question can have many valid answers, and failure modes are subtle and hard to predict.
- → An LLM application is never truly finished — there is always a further prompt optimisation available.

The regression you cannot see

A customer reports that billing tickets are misrouted. You add a line to the system prompt emphasising billing keywords, confirm the reported ticket now routes correctly, and ship. Two weeks later, technical tickets containing the word 'charge' — as in 'my device will not charge' — are being sent to billing. Nothing alerted you, because you only ever re-checked the case you were fixing.

02 The three core challenges of LLM development

4:52

> Understanding your data, closing the specification gap, and coping with inconsistent behaviour — these are what evaluation exists to address.

Drilling into why this is hard produces three distinct sub-problems.

First, understanding the data. When you build a workflow you test isolated, simple examples — the happy path, the way you imagine users will behave. At scale it is drastically different, because users are unpredictable. You genuinely do not know what people are asking, how your system is responding, or what the edge cases are until you look.

Second, the specification gap. There is always a distance between what you want the system to do and what you can actually express in prompts and code. You know a good output when you see it; translating that recognition into clear instructions is surprisingly hard. And when you spot a bad answer, attributing it is its own problem — was it a prompting issue, a retrieval issue, a combination, or an error in your pipeline logic? You need to be able to tell those apart to close the gap.

Third, inconsistent behaviour. The system works on your test cases and behaves unpredictably on real inputs. Small changes in wording lead to completely different outputs, which makes the product feel unreliable. Subtle differences in how a question is phrased or a document is formatted can change everything.

Because of these, success depends on how fast you can iterate — and that in turn rests on three capabilities: evaluation quality (systematic measurement through automated tests, human evaluation, and success metrics), debugging ability (trace logging, data inspection, error analysis), and the ability to change behaviour based on what you learned. Most teams only build the third, which is exactly why they never improve past the demo. All three together create a virtuous improvement cycle.

KEY POINTS

- → Understanding the data: you test the happy path; real usage at scale is drastically different and users are unpredictable.
- → The specification gap: you recognise a good output but cannot fully express what makes it good in a prompt.
- → Attribution is part of the gap — was the failure prompting, retrieval, or pipeline logic?
- → Inconsistent behaviour: small wording changes produce completely different outputs.
- → Success rests on three capabilities — evaluation quality, debugging, and the ability to change behaviour. Most teams build only the last.

Attribution in a RAG pipeline

A customer gets a wrong answer about a refund window. There are at least three distinct causes with three distinct fixes, and you cannot tell them apart without instrumenting each stage.

```
Bad answer observed
  |
  +-- Retrieval returned the wrong policy document?
  |     -> fix chunking / embeddings / reranking
  |
  +-- Right document retrieved, model ignored it?
  |     -> fix the prompt's grounding instructions
  |
  +-- Right document, right reading, wrong arithmetic?
        -> move the computation out of the model
```

03 What an eval is, and the analyze-measure-improve cycle

9:01

> An eval is a systematic measurement of one specific quality aspect; the working loop is collect failures, turn them into metrics, then improve.

The definition is deliberately plain: an evaluation is a systematic measurement of quality in an AI system. An individual eval is a single metric measuring one specific aspect of performance — and one system has many of them. You might separately check responses for factual accuracy, tone appropriateness,

instruction-following, and output-format compliance.

Evals get operationalised in three ways. As background monitoring, tracking performance over time. As guardrails, blocking bad outputs in real time. And as improvement tools, labelling data for fine-tuning or building custom test suites.

The lifecycle for applying them — a mental model drawn from work by Hamel Husain and Shreya Shankar on application-centric AI evals for engineers and technical PMs — is analyze, measure, improve. Analyze: collect examples of what is flowing through your system and categorise the failure modes. In the real world these arrive as user complaints, errors you spot while reading data, or outright application errors like crashes and outputs that failed to store. Measure: translate those insights into quantitative metrics, which can be as simple as a boolean pass/fail, or scores and rankings on a 0–1 or 1–10 scale. Improve: refine prompts, try different models, change the architecture.

Then you go round again. The cycle is what addresses all three core challenges at once, and running it repeatedly is what creates a flywheel of continuous improvement rather than a series of one-off fixes.

KEY POINTS

- → An eval is a systematic measurement of quality; a single eval measures one specific aspect, and a system has many.
- → Three operationalisations: background monitoring, real-time guardrails, and improvement tooling for labelling and datasets.
- → Analyze: collect real examples and categorise failure modes from complaints, data inspection, and application errors.
- → Measure: turn insights into quantitative metrics — booleans first, scores and rankings later.
- → Improve: refine prompts, swap models, change architecture — then repeat the cycle.

One system, many evals

A single support workflow carries several independent measurements, each answering a different question about the same response.

```
evals = {
    "format_valid":     lambda r: r.category in CATEGORIES,
    "has_response":     lambda r: len(r.response) > 10,
    "factually_correct": judge_factual,     # LLM judge
    "tone_appropriate":  judge_tone,        # LLM judge
    "no_pii_leaked":     regex_pii_check,   # guardrail
}
```

04 Level 1: unit tests as evals

12:26

> Fast, cheap Python assertions over structured output, run on every code and prompt change.

Level 1 is the cheapest tier and should run constantly — on every code change and every prompt change. These are ordinary unit tests, the same kind any serious software project has, applied to an LLM workflow.

The mechanism is Python's assert. Everything after the keyword must evaluate to true; if it does, execution continues silently, and if it is false, Python raises an AssertionError you can catch and report.

The most productive place to apply this is structured output, because structured output gives you fields with types and allowed values that you can assert against deterministically. Take a support ticket: 'My credit card was charged twice this month.' Your workflow classifies it and returns a category plus a confidence score. You can now assert that the category is one of your known categories, that the confidence is a float, that it lies between 0 and 1, and — specific to this input — that the category is billing.

Organise these as an evals folder containing raw event JSON captured from your actual system. This is why capturing raw events matters so much: those files become your test fixtures. As your workflow grows to five or ten steps, you can assert at every step — the classification, the routing decision, the confidence on the structured object, the generated answer, the success flag from the final API call.

What should you test? The failure modes. When you hit an error and diagnose it, write a test that pins it down so it can never recur silently. Start with a simple loop and a print-based report; move to pytest when you want a proper runner.

KEY POINTS

- → Fast, cheap assertions run on every code and prompt change — the tier you run most often.
- → Python's assert passes silently on true and raises AssertionError on false.
- → Structured output is the natural target: assert on allowed values, types, and ranges.
- → Keep an evals folder of raw event JSON captured from your real system — this is why capturing raw events matters.
- → Assert at every step of a multi-step workflow, not just the final output.
- → What to test comes from your failure modes: when something breaks, write the test that catches it next time.

Assertions over a classification step

Load a real captured event, run the workflow, and assert the properties that must hold. The generic checks catch schema drift; the specific check pins this ticket's expected routing.

```
import json
from pathlib import Path

CATEGORIES = {"billing", "technical", "general"}

def load_event(name):
    return json.loads(Path(f"evals/events/{name}.json").read_text())

def test_billing_categorization():
    event = load_event("billing_double_charge")
    result = process_customer_message(event["message"])

    # generic invariants - catch schema drift
    assert result.category in CATEGORIES
    assert isinstance(result.confidence, float)
    assert 0.0 <= result.confidence <= 1.0
    assert len(result.response) > 10

    # input-specific expectation
    assert result.category == "billing"
```

A minimal test runner

Before reaching for pytest, this is enough to get value. It reports which scenarios passed and shows the assertion that failed.

```
TESTS = [
    test_billing_categorization,
    test_feature_request_categorization,
    test_support_categorization,
]

passed = 0
for test in TESTS:
    try:
        test()
        print(f"PASS  {test.__name__}")
        passed += 1
    except AssertionError as e:
        print(f"FAIL  {test.__name__}: {e}")

print(f"\n{passed}/{len(TESTS)} tests passed")
```

Asserting through a multi-step workflow

As the cognitive architecture grows, each stage gets its own assertion inside the same test, so a failure localises to a step.

```
def test_billing_end_to_end():
    event = load_event("billing_double_charge")

    classified = classify(event)
    assert classified.category == "billing"

    routed = route(classified)
    assert routed.handler == "billing_handler"

    answer = generate(routed)
    assert answer.confidence > 0.5
    assert "refund" in answer.response.lower()

    sent = deliver(answer)
    assert sent.success is True
```

05 Level 2: humans first, then an aligned model judge

17:55

> The common misconception is that you can hand evaluation straight to an LLM. You cannot instruct a model to review what you have not learned to review yourself.

Level 2 is systematic review of quality, run weekly or bi-weekly depending on system size. It is where most of the value lives and where the most common mistake is made.

The misconception is that you plug in an eval tool, give an LLM some instructions, and let it monitor your whole system. That is the dream, and it is what you should avoid at all cost in the beginning. The reason is straightforward: how can you instruct a model to perform a review when you have no idea what to look for or how to score these inputs and outputs? Your eval must be aligned with how a human — ideally a domain expert — reads the data.

So humans come first. Look at as much data as possible initially, and involve both an engineer, who needs to understand what is flowing through the system, and domain experts, who know what good means in the business context. Remove all friction from viewing data: build simple dashboards, or just use a spreadsheet, which goes a long way. Save good examples for later reference.

Only once you have a feel for how a human evaluates your system do you turn to an LLM judge. Use the most capable model you can afford — this is not the job for a cheap small model. Focus on specific quality dimensions rather than a single overall score, start with binary good/bad, and always generate detailed critiques rather than bare scores, so the model explains why.

And this never fully ends. Your system and data will drift over time — through prompt changes, data changes, new features, different users — so what 'good' means shifts too. Human and model evaluation go hand in hand permanently; you track human-model agreement over time rather than declaring the judge finished after two weeks.

KEY POINTS

- → Run systematic review weekly or bi-weekly; it costs more than level 1 because humans spend time on it.
- → You cannot skip the human step — you cannot write instructions for a review you have not learned to perform.
- → Involve both engineers (to understand the data flow) and domain experts (to know what good means).
- → Remove all friction from viewing data; a spreadsheet is a legitimate and effective tool.
- → Use the most capable model you can afford as the judge, focus on specific dimensions, and start binary.
- → Always generate detailed critiques, not just scores.
- → Drift is permanent — what good looks like changes, so track human-model agreement continuously.

A judge prompt with a required critique

Binary outcome, explicit criteria, and an explanation the human can audit. Start here rather than with a five-metric scoring rubric.

```
JUDGE = """Evaluate this customer service response on these criteria:

- accuracy:    are the facts correct?
- helpfulness: does it solve the customer's problem?
- tone:        is it professional and empathetic?

<question>{question}</question>
<response>{response}</response>

Provide a detailed critique explaining your reasoning,
then rate 1 (good) or 0 (bad).
"""

class Critique(BaseModel):
    explanation: str
    outcome: bool
```

06 The alignment process and the agreement score

22:47

> Run model critique and human critique in parallel over the same rows, score the match, and iterate the judge prompt until agreement climbs.

This is the most fundamental component of improving an LLM application over time, and almost everyone skips it.

The process: collect model predictions, generate model critiques, get human evaluations on the same data, compare model versus human judgements, iterate on the evaluator prompt, and repeat until agreement is sufficient.

Concretely, it is a spreadsheet. Column A holds inputs — ideally real questions captured from your system, though synthetic data generated by an LLM is a fine starting point if you do not yet have a hundred real ones. Column B holds the output your workflow produced for each. Column C is the model critique: loop the input-output pairs through your judge prompt with structured output, storing both the explanation and a boolean outcome. In parallel — and this is the essential part — a human reviews the same pairs and records their own critique and outcome. A final column checks whether the two match: 1 if the human and model outcomes agree, 0 if they do not.

The mean of that column is your agreement score. In the worked example it starts at 70% across ten rows. Whether that is good depends on your system, but you want it as high as you can drive it.

Now iterate — and note carefully that the human critique stays fixed. It is your golden standard, the thing you are optimising toward. What changes is the judge prompt. You can tweak it by hand, or use meta-prompting, which is the more powerful option: export the dataset, show a strong model the inputs, outputs, human critiques, model outcomes, the current agreement score, and the current judge prompt, then ask it to rewrite the prompt to maximise agreement. The model spots patterns in what the human cared about and folds them into the judge prompt. Re-run, record the new agreement score, and repeat.

This is much closer to how data scientists and ML engineers work than to ordinary software engineering: continuous experiments, moving a metric by percentage points, requiring you to understand the nuances of what is actually happening in your system.

KEY POINTS

- → The loop: model critiques and human critiques on the same rows, compared, with the judge prompt iterated until they agree.
- → The human critique is the fixed golden standard; the judge prompt is what you change.
- → Agreement score = the fraction of rows where the human and model outcomes match.
- → Meta-prompting — showing a strong model the mismatches and the current judge prompt, and asking it to rewrite for higher agreement — is the powerful way to iterate.
- → Real captured inputs are best, but LLM-generated synthetic examples are a legitimate starting point.
- → This resembles ML experimentation more than software engineering: move a metric through repeated experiments.

The alignment spreadsheet

Ten rows, two independent critiques, and one derived column. The 70% at the bottom is the number every iteration tries to raise.

```
| # | input              | output          | model | human | match |
|---|--------------------|-----------------|-------|-------|-------|
| 1 | order 3 days late  | I understand... | 1     | 1     | 1     |
| 2 | cancel subscription| Sure, here...   | 1     | 1     | 1     |
| 3 | charged twice      | Let me check... | 1     | 0     | 0     |
| 4 | where is my refund | Refunds take... | 0     | 0     | 1     |
| 5 | wrong size sent    | Apologies...    | 1     | 1     | 1     |
|...|                    |                 |       |       |       |

agreement score = 7/10 = 70%
```

A human critique that the model missed

Row 3 is where the judge and the domain expert diverge, and the human's reasoning is exactly the signal meta-prompting needs to fold into the next judge prompt.

```
Human critique (outcome: BAD):
  "This response is too casual for a 3-day delay. The customer
   said 'late', which implies frustration. It should acknowledge
   that the delay is unacceptable, show more urgency, and
   proactively offer to discuss compensation."

Model critique (outcome: GOOD):
  "The response is polite, acknowledges the issue, and asks for
   the order number to investigate further."

-> The judge has no notion that delay severity should change tone.
```

The meta-prompt that rewrites your judge

Rather than guessing at prompt edits, hand the discrepancies to a strong model and let it find the pattern in the human's preferences.

```
Below are 10 input/output pairs from our support system.

For each I have included:
  - the human expert's critique and outcome (the gold standard)
  - our current LLM judge's critique and outcome

Current agreement: 70%.

Here is the judge prompt we are currently using:
<current_prompt>{judge_prompt}</current_prompt>

Study the rows where the judge disagreed with the human. Identify
what the human consistently cares about that the judge misses.
Rewrite the judge prompt so that re-running it on this data would
reach as close to 100% agreement as possible. Do not overfit to
individual rows - encode the general principle.
```

Computing the agreement score

The metric itself is trivial. The work is in producing the two columns honestly.

```
def agreement_score(rows):
    matches = sum(1 for r in rows
                  if r["human_outcome"] == r["model_outcome"])
    return matches / len(rows)

for i, prompt in enumerate(judge_prompt_versions, 1):
    rows = run_judge(dataset, prompt)
    print(f"iteration {i}: {agreement_score(rows):.0%}")

# iteration 1: 70%
# iteration 2: 80%
# iteration 3: 90%
```

07 Level 3: A/B testing with real users

31:46

> The trickiest and most expensive level — compare prompts, models, or whole workflows by their effect on real business outcomes.

Level 3 runs experiments with real users to measure business impact, reserved for major releases because of its cost. You use it to compare different prompts, different models, or entirely different workflows.

The cost is high because you need enough data for the comparison to mean anything. Running five examples and concluding 'I think option B is better' is not an experiment. You want volume, which takes time and exposes real users to the variant.

There is a practical middle ground worth knowing. You can run the comparison internally against your own evaluation harness — compare two system prompts by the overall good/bad rate from your human column, or later from your aligned judge — without wiring up a full user-satisfaction system. In the worked example the human scores give a 60% good rate, and a competing prompt can be measured against that same set. This gives you most of the decision value at a fraction of the cost.

Full A/B testing embedded in the application, with users giving satisfaction ratings or thumbs up/down, is genuinely hard to implement, and it only really makes sense once your application is mature and has enough users that small prompt changes matter. The metrics at this level are business metrics: user satisfaction score, task completion rate, time to resolution, engagement, and ultimately sales and retention.

KEY POINTS

- → Level 3 measures business impact with real users, reserved for major releases because it is costly.
- → It requires enough data to be meaningful — a handful of examples is not an experiment.
- → A cheaper middle ground: compare prompt variants against your own labelled set and its good/bad rate.
- → Full in-app A/B testing with satisfaction ratings is complex and suits mature, high-traffic applications.
- → Metrics here are business metrics: satisfaction, task completion, time to resolution, engagement, retention.

Offline prompt comparison before any user sees it

Run two prompt variants over the same fixed dataset and compare their good rates using the judge you already aligned. Most decisions can be made here.

```
def good_rate(prompt, dataset, judge):
    outputs = [run_workflow(row["input"], prompt) for row in dataset]
    verdicts = [judge(o) for o in outputs]
    return sum(v.outcome for v in verdicts) / len(verdicts)

baseline = good_rate(PROMPT_V1, dataset, aligned_judge)  # 0.60
variant  = good_rate(PROMPT_V2, dataset, aligned_judge)  # 0.75

print(f"baseline {baseline:.0%} -> variant {variant:.0%}")
```

08 Reference-based versus reference-free metrics

34:29

> When you know the correct answer, testing is easy; when many answers are valid, you score qualities instead.

Metrics divide into two categories, and knowing which one you are in tells you how hard the measurement will be.

Reference-based metrics apply when you know the golden answer — you know what the output should be, so you compare against it. Exact string matching, semantic similarity, code execution results, SQL query correctness, and structured data validation all fall here. These are comparatively easy to implement, and much of this can be captured at level 1 with unit tests.

Reference-free metrics apply when you do not know exactly what the correct output should be, or when many outputs are legitimately correct. Answering a customer query has effectively infinite good answers, differing in tone, length, and structure. Here you score qualities instead: tone appropriateness, length constraints, absence of hallucination, format compliance, safety and toxicity. These are typically scored on a scale — 0 to 1, or 1 to 5 — where the top of the scale means a perfect match to how you feel the output should be.

In the real world you will find yourself working with reference-free metrics a great deal, which is precisely why the LLM-as-a-judge machinery and its alignment process matter so much. Whichever category you are in, the advice is the same: start simple, start binary, good and bad. Getting that working is challenging enough.

KEY POINTS

- → Reference-based: you know the correct answer — exact match, semantic similarity, code execution, SQL correctness, schema validation.
- → Much reference-based checking can live at level 1 as unit tests.
- → Reference-free: many outputs are valid — tone, length, hallucination, format compliance, safety and toxicity.
- → Reference-free metrics are usually scored on a 0–1 or 1–5 scale rather than pass/fail.
- → Real systems lean heavily on reference-free metrics, which is why judge alignment matters.
- → Whichever you use, start binary — good and bad is hard enough to get right.

Both kinds on one response

The same support reply carries reference-based checks that are trivially automatable and reference-free ones that need a judge.

```
# reference-based - a known correct value exists
assert result.category == "billing"
assert result.order_id == "A-10294"
assert json.loads(result.payload)          # schema valid

# reference-free - many valid answers, score the quality
tone      = judge_tone(result.response)        # 0.0 - 1.0
grounded  = judge_no_hallucination(result)     # bool
safe      = judge_safety(result.response)      # bool
```

09 Four mistakes, and the principles that avoid them

37:12

> Tool-first thinking, generic-metrics obsession, avoiding your data, and unaligned judges — with looking at data as the highest-value habit.

Four failure patterns recur, each with a direct fix.

Tool-first thinking is reaching for a purchase instead of a diagnosis. RAG problems? Let us buy a new vector database. Accuracy issues? Let us switch to a more powerful model. Need a metric? Let us buy an eval platform with ten metrics out of the box. The fix is to start simple and build custom solutions based on what you actually need.

Generic-metrics obsession is drowning in meaningless scores — helpfulness 4.2, truthfulness 4.5. What does that tell you? Nothing actionable. The fix is specific, actionable metrics built from what you want to know about your system, starting with booleans.

Avoiding your data is the most consequential. It sounds like: we can use a tool for this, the LLM will check it, real users will tell us if it is broken, trust your gut, go with vibes. The fix is to look at lots of real data constantly. A principal engineer at Zapier reportedly spends three hours a day looking at traces and data — someone paid to write software who understands you cannot improve an AI system without knowing what flows through it. Engineers from a software background ignore this most often; it is closer to how data scientists work.

Unaligned LLM judges is assuming the judge works without validating it. The fix is always to validate judge alignment first by bringing a human into the loop.

The principles that follow: start simple and specific, focusing on your biggest problems first. Use existing tools before buying new ones. Build domain-specific evaluations, because they differ for every project. Start with unit tests and manual review — that gets you a long way. Look at lots of data; manual inspection has the highest value-to-prestige ratio, and you are doing it wrong if you are not looking at data. Sample broadly, then focus on patterns, and never stop examining real examples. Remove all friction from viewing data, build custom tools for your domain, automate repetitive tasks, and track metrics over time. And only use LLMs to scale — generating test cases, creating synthetic data, automating critique and labelling — but always validate with humans.

KEY POINTS

- → Tool-first thinking: buying a database, a model, or a platform instead of diagnosing. Fix: start simple, build what you need.
- → Generic-metrics obsession: 'helpfulness 4.2' is unactionable. Fix: specific metrics, starting binary.
- → Avoiding your data: the most consequential mistake. Fix: look at lots of real data, constantly and forever.
- → Unaligned judges: assuming the judge works. Fix: validate alignment with a human first.
- → Manual inspection has the highest value-to-prestige ratio — unglamorous and the most useful thing you can do.
- → Only use LLMs to scale: generate test cases, create synthetic data, automate critique — but always validate with humans.

You are succeeding versus you are struggling

The concrete symptoms of an evaluation-driven workflow, contrasted with its absence. This is the diagnostic to run on your own team.

```
Succeeding                        Struggling
--------------------------------  --------------------------------
Deploy changes confidently        Everything breaks something else
Failures caught before users      Surprised by user complaints
You understand system behaviour   Progress feels like trial & error
Improvements compound over time   You cannot measure if changes help
```

The full development cycle

How the levels fit together in day-to-day work: monitor, capture, isolate, fix locally, then re-run everything.

```
1. Monitor traces / failure patterns / user feedback
2. Capture the raw event into evals/events/
3. Write a test that reproduces the failure   (level 1)
4. Fix locally - prompt, model, or architecture
5. Confirm the new test passes
6. Re-run ALL tests - did the fix break anything?
7. Weekly: human + judge review on fresh data  (level 2)
8. Major release: A/B test the change          (level 3)
```

Glossary

Eval

A systematic measurement of quality in an AI system; a single eval measures one specific aspect of performance.

Evaluation-driven workflow

A development process where improvement is systematic rather than accidental, because every change is measured against a fixed set of evals.

Analyze–measure–improve

The evaluation lifecycle: collect and categorise failures, turn insights into quantitative metrics, then refine prompts, models, or architecture.

Specification gap

The distance between what you want a system to do and what you can actually express in prompts and code.

Assertion

A Python statement that passes silently when its condition is true and raises an AssertionError when false, used as the basis of level 1 evals.

Agreement score

The fraction of examples where a human's critique outcome matches the LLM judge's outcome; the metric you iterate the judge prompt to improve.

Meta-prompting

Asking a strong model to rewrite your judge prompt by showing it the human-model disagreements and the current prompt, so it encodes what the human cares about.

Reference-based metric

A measurement made against a known correct answer — exact match, semantic similarity, code execution, or schema validation.

Reference-free metric

A measurement of a quality where many outputs are valid — tone, length, hallucination, format compliance, safety.

Guardrail

An eval operationalised in real time to block a bad output before it reaches the user.

Raw event

The exact input payload your workflow received, captured and stored so it can later be replayed as a test fixture.

Drift

The gradual change over time in what 'good' means for your system, caused by prompt changes, data changes, new features, or different users.

Check yourself

Answer first, then open each one.

Why can you not build an LLM judge before doing human evaluation, even though the judge would save time?

Because the judge prompt is a written specification of what to look for, and you cannot write that specification for a review you have not learned to perform. Until a human — ideally a domain expert — has read real data and formed a view of what good means, you have nothing to instruct the model with, and any metrics it produces will look rigorous while measuring the wrong thing.

During the alignment loop, why does the human critique stay fixed while the judge prompt changes?

Because the human critique is the golden standard you are optimising toward. If you adjusted the human labels to match the model, you would be moving the target to meet the measurement, which guarantees agreement while destroying its meaning. Only the judge prompt is a free variable.

Why is capturing raw events from production described as crucial for level 1 evals?

Because raw events are your test fixtures. Assertions need realistic inputs, and inputs you invent reflect the happy path you already imagined rather than the messy reality users produce. Stored raw events let you replay exactly what your system received when it failed, turning each incident into a permanent regression test.

Your judge reports 'helpfulness 4.2' across all outputs. Why is this an example of generic-metrics obsession, and what would be better?

Because the number is not actionable — it does not tell you what is wrong or what to change, and small movements in it are indistinguishable from noise. A better approach is specific binary metrics tied to failure modes you have actually observed, such as 'did it give the customer a route to resolution?', which points directly at a fix when it fails.

Why does the lesson say human and model evaluation must continue permanently rather than being finished after a few weeks of alignment?

Because of drift. Prompt modifications, data changes, new features, and different users all change what good looks like over time, so a judge aligned to last quarter's standard slowly stops measuring the current one. Tracking human-model agreement continuously is what detects that divergence before it corrupts your metrics.

You fix a misrouted billing ticket by editing the system prompt and confirm the fixed case now works. Why is this insufficient, and what does the three-level structure add?

Because confirming the case you fixed says nothing about regressions elsewhere — the prompt change may have broken cases you are not looking at. Level 1 gives you a suite of assertions over captured events that you re-run after every change, so 'I fixed it' becomes 'I fixed it and the other cases still pass'.

Where to go next

- → Create an evals/events/ folder, drop in ten real captured payloads from your system, and write assertions for the structured fields each one should produce.
- → Run the alignment exercise on ten rows: judge critique in one column, your own critique in another, and compute the agreement score.

- → Take your worst-disagreeing rows and write the meta-prompt that rewrites your judge prompt, then measure whether agreement actually improved.
- → Convert your ad-hoc test loop to pytest so the whole eval folder runs from a single terminal command.
- → Audit your current metrics: for each one, ask what you would change if it moved by 10%. Delete the ones with no answer.
- → Block out time to read production traces with no goal other than understanding what users actually send you.
